DeepStream human pose recognition and action classification

DeepStream human pose recognition and action classification

1. Case goal and processing chain
2. NVIDIA official deployment method (only for reference when restoring the environment)
3. Course script entry and output rules
4. Pose classification with two local videos
   - 4.1 Run
   - 4.2 Replace the videos
   - 4.3 Video output
5. Real-time pose classification with the USB camera
   - 5.1 Run directly
   - 5.2 Preliminary camera check
6. Real-time pose classification with the CSI camera
   - 6.1 Run directly
   - 6.2 Common CSI problems
7. Description of the current implementation
8. Stopping and troubleshooting

> Applicable environment: JetPack 7.2.1, Ubuntu 24.04, DeepStream 9.1, CUDA 13.2, Jetson Orin NX.

> The factory system has already completed the model download, the TensorRT engine build, the application compilation and the deployment of the course entry. For normal study, start running directly from section 3; refer to section 2 to restore the environment only when the program, the models or the engines are missing.

1. Case goal and processing chain

This case is based on the NVIDIA official `deepstream-pose-classification`. It detects the people in the frame, draws the human skeleton, and determines the behaviour category of a person from the consecutive actions.

The complete processing chain is as follows:

text

```
Input video or camera
        ↓
PeopleNet: detect people
        ↓
NvDCF Tracker: assign and maintain track IDs for the people
        ↓
BodyPose3DNet: output 34 2D/3D body key points
        ↓
Pose post-processing: draw the skeleton
        ↓
Temporal preprocessing: accumulate 300 frames of key points per track ID
        ↓
PoseClassificationNet (ST-GCN): output the action category
```

The red box in the frame is the people detection box, the coloured line segments and dots are the human skeleton, and text such as `person 3 walking` inside the box means: `person 3` is the target ID of the Tracker, and `walking` is the current action classification result of that target.

This case is not "single image classification". The input of the action classification model is temporal data of **300 consecutive frames** with **34 key points per frame**. Therefore, when a person has just appeared and the key point sequence has not yet accumulated enough, there may be only the people box and the skeleton at first, and the stable action label appears afterwards. With a 30 FPS input, 300 frames correspond to about 10 seconds of action history.

2. NVIDIA official deployment method (only for reference when restoring the environment)

Official source code and README: https://github.com/NVIDIA/DeepStream/tree/main/src/apps/tao_apps/apps/tao_others/deepstream-pose-classification

Source code directory:

```
~/deepstream_tao_apps/apps/tao_others/deepstream-pose-classification
```

When restoring the environment for the first time, you need Eigen, the CUDA NVTX headers, and the PeopleNet, BodyPose3DNet, PoseClassificationNet and ReIdentificationNet models. The CUDA version of the current system is 13.2, so the compilation command must specify the version explicitly:

bash

```
# Create it only when /usr/include/Eigen does not exist; the symlink persists and survives a
reboot
sudo ln -sf /usr/include/eigen3/Eigen /usr/include/Eigen

# Install only when nvtx3/nvToolsExt.h is missing
sudo apt install cuda-nvtx-13-2

cd ~/deepstream_tao_apps/apps/tao_others/deepstream-pose-classification
export CUDA_HOME=/usr/local/cuda CUDA_VER=13.2 DS_ROOT=/opt/nvidia/deepstream/deepstream-9.1
make clean && make
```

On the first run, DeepStream may need to build the engine according to the GPU, CUDA and TensorRT versions of the machine, which takes a noticeably longer time. The current course environment has already generated and verified the required engines; do not casually delete the `.engine` files under the `models/` directory.

3. Course script entry and output rules

The current unified startup script is located in `~/deepstream_tao_demo/`:

```
cd ~/deepstream_tao_demo
```

Input source	Pose classification (with the action labels)	Pure 3D skeleton (without the action)
Local video	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh`	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_bodypose3d.sh`
USB camera	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh usb`	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_bodypose3d.sh usb`
CSI camera	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh csi`	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_bodypose3d.sh csi`

The real-time window is displayed on the local Jetson desktop or on the VNC desktop. The pose classification uses the PeopleNet → Tracker → BodyPose3DNet → PoseClassificationNet chain; the pure 3D skeleton turns off PoseClassificationNet and only shows the skeleton.

The pure 3D skeleton entry uses `bodypose_only.yaml`, `bodypose_usb_noclass.yaml` and `bodypose_csi_noclass.yaml`. The YAML turns off both `display-text` and `display-bbox`, and the application keeps the skeleton lines generated by the pose post-processing, without showing the people boxes, the action text or the frame number. This behaviour depends on the recompiled `deepstream-pose-classification-app`; after updating the source code, run `~/deepstream_tao_demo/build_deepstream_apps.sh`.

When stopping early in any mode, press `Ctrl+C` only once, then wait for `End of stream` to appear in the terminal and for the program to exit automatically. Do not press `Ctrl+C` repeatedly, do not close the terminal directly and do not use `kill -9`.

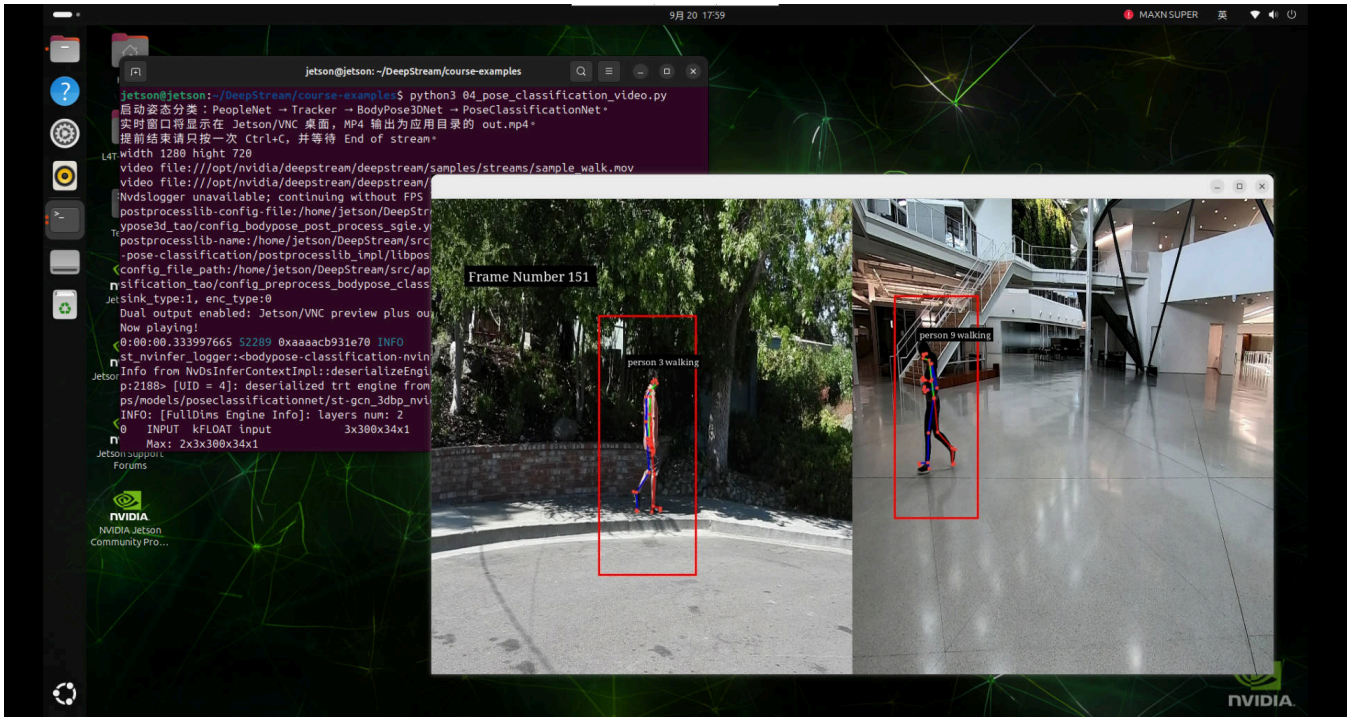
4. Pose classification with two local videos

4.1 Run

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh
```

The current course YAML uses the two inputs that correspond to the tutorial by default:

```
source-list:
  list:
file:///opt/nvidia/deepstream/deepstream/samples/streams/sample_walk.mov;file:///opt/nvidia/
deepstream/deepstream/samples/streams/sample_run.mov
```



By default, the preview window has one video on the left and one on the right, which show the people boxes, the Tracker ID, the skeleton and the action classification result; the Tiler composes the two results into a single window.

4.2 Replace the videos

Edit the `source-list.list` of the following file:

```
~/deepstream_tao_apps/configs/app/bodypose_full.yaml
```

The two videos are separated by a semicolon, for example:

```
source-list:
  list: file:///home/jetson/videos/a.mp4;file:///home/jetson/videos/b.mp4
```

When only one URI is kept, the program switches to single file input. The path must be an absolute path on the Jetson; it is recommended to use MP4/H.264 material that DeepStream decodes easily.

You can also override it directly by passing an argument:

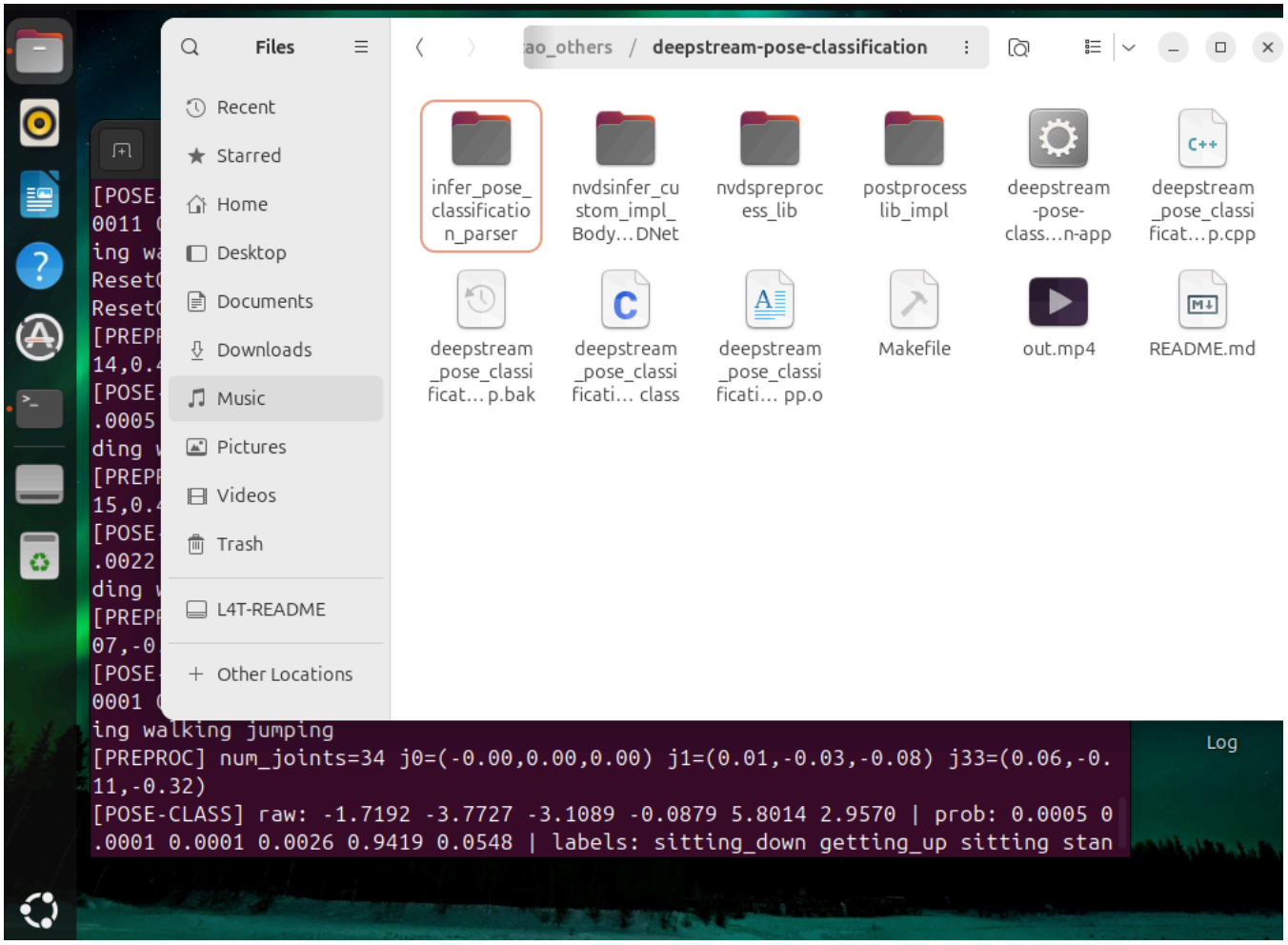
```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh
/home/jetson/Videos/test.mp4
```

4.3 Video output

The video mode previews in real time and at the same time saves the complete MP4 with the detection boxes, the skeleton and the action labels:

```
~/deepstream_tao_apps/apps/tao_others/deepstream-pose-classification/out.mp4
```

The current implementation sends EOS on the first `Ctrl+C` and exits after the MP4 has finished writing its index. Only after the normal playback ends or the EOS finalization completes is `out.mp4` a complete, playable file.



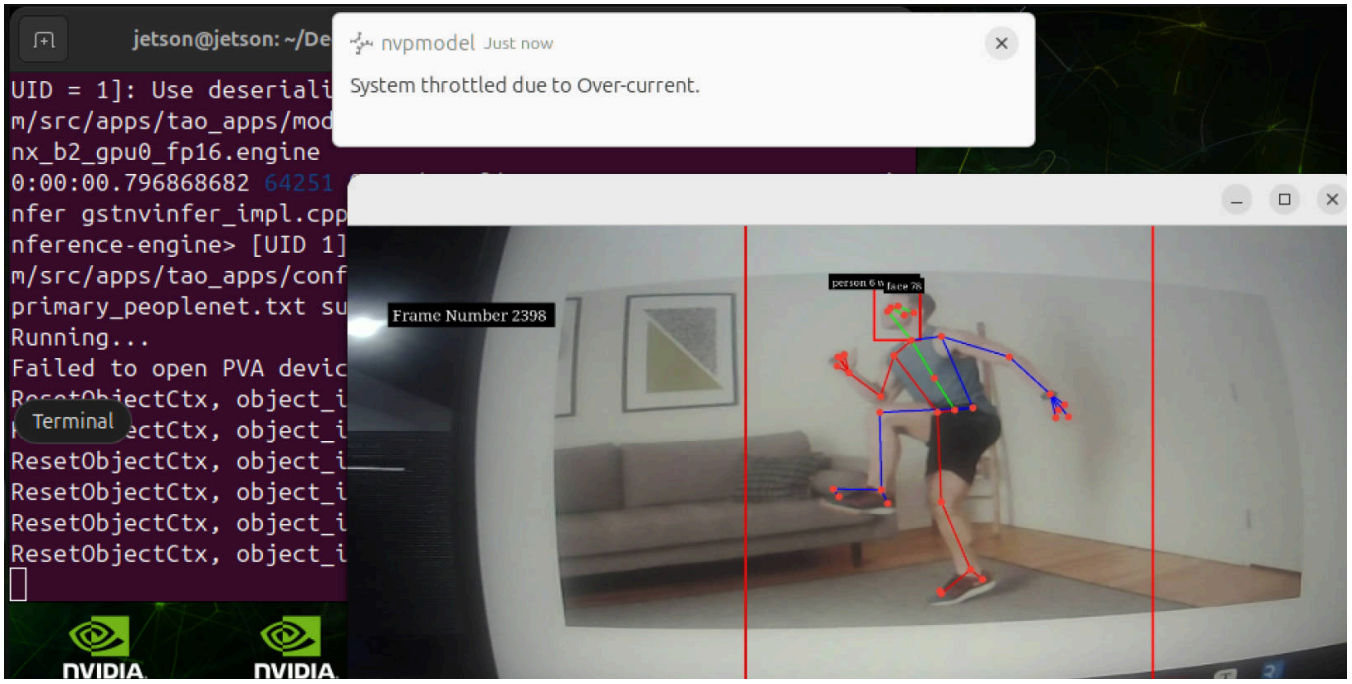
5. Real-time pose classification with the USB camera

5.1 Run directly

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh usb
```

The default parameters fit the USB UVC cameras that are commonly used at the moment:

```
Device: /dev/video0
Format: YUY2
Resolution: 640×480
Frame rate: 15 FPS (USB 2.0 bandwidth limit in YUYV mode)
```



The USB mode only previews in real time; it forcibly uses the display sink and does not create or rewrite the `out.mp4` in the pose classification application directory.

5.2 Preliminary camera check

```
v4l2-ctl -d /dev/video0 --list-formats-ext
# Confirm the device number, the output format and whether it is in use
```

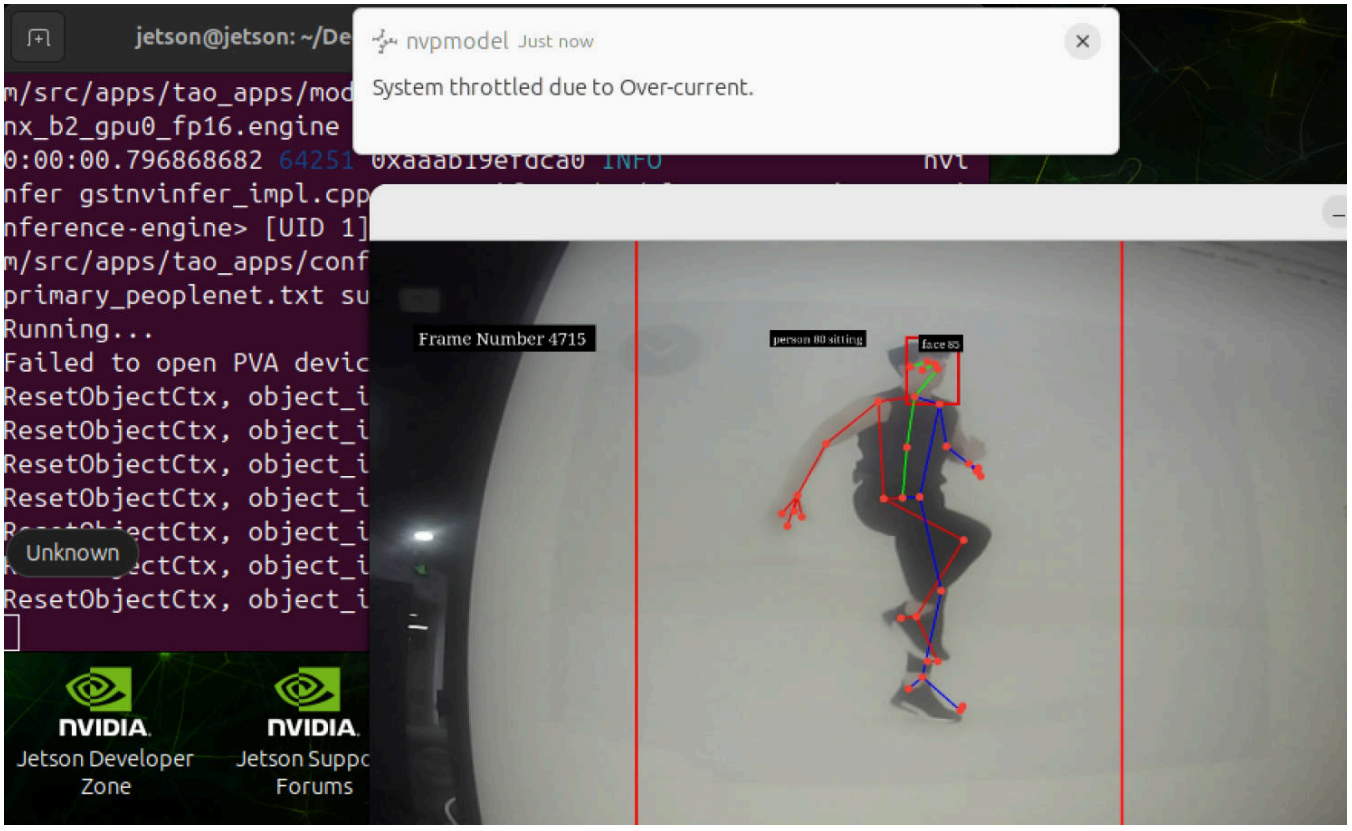
The current course entry uses the native YUY2 format by default. If the camera only supports MJPEG, you must first confirm the actual format and then add a dedicated decoding adaptation for MJPEG; do not simply fill MJPEG in as YUY2.

6. Real-time pose classification with the CSI camera

6.1 Run directly

Run:

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_poseclass.sh csi
```



In CSI mode, the script automatically starts the relay from the native Jetson `nvarguscamerasrc` to `/dev/video10`, and the default parameters are:

```
sensor-id: 0
sensor-mode: 3 (selected from the 1920×1080@30 caps when the plugin has no such property)
wbmode: 1 (auto white balance)
exposurecompensation: 0
Resolution: 1920×1080
Frame rate: 30 FPS
```

6.2 Common CSI problems

If the terminal reports `No cameras available`, it means that Argus currently does not detect the CSI sensor; you should check the ribbon cable, the camera module, the `sensor-id` and the state of the `nvargus-daemon`:

```
systemctl status nvargus-daemon
~/deepstream_tao_demo/csi-mode status
~/deepstream_tao_demo/csi-mode direct
```

7. Description of the current implementation

The official example originally creates the input source from the file URI in the YAML. The course version adds two kinds of camera sources without modifying the video YAML:

```
USB: v4l2src → videoconvert → nvvideoconvert → NVMM
CSI: the script automatically runs the csi-mode writer, via v4l2loopback → v4l2src → NVMM
```

The USB/CSI entry selects the camera source through an argument and forcibly overrides the output policy to "preview only"; the local video entry keeps "preview + MP4 recording". This is the reason why the three scripts share the same main program but have a different output behaviour.

In order for the pose classification model to receive the upstream data correctly, the course environment also fixes two configuration connections of the official example:

1. PeopleNet uses the GIE ID `1`, BodyPose3DNet/the post-processing use the ID `2`, and PoseClassificationNet classifies the body pose results with the ID `2`.
2. The Tracker uses the ReIdentificationNet TensorRT engine that already exists in the current environment, which avoids referencing the `.etlt` model missing in the old version.

These are already fixed in the current environment; no manual change is needed when you use the course entry normally.

8. Stopping and troubleshooting

Symptom	What to check first
No people box and no skeleton	Whether the person appears completely in the frame, whether the size is large enough and whether the lighting is too dark.
There is a skeleton but no action text for the time being	The 300-frame key point sequence of the same person has not been fully accumulated yet; keep the target continuously visible in the frame.
The USB reports `Device ... is busy`	Another program is using that `/dev/video*`; close the program that occupies the camera and then run it again.
The USB shows no image or the negotiation fails	Use `v4l2-ctl -d /dev/video0 --list-formats-ext` to verify the device number, the format, the resolution and the frame rate.
The CSI reports `No cameras available`	Check the CSI module, the ribbon cable, the `sensor-id` and the `nvargus-daemon`.
The CSI shows no image	Check `~/deepstream_tao_demo/csi-mode status` and `/tmp/deepstream_tao/csi_writer.log`; when the script starts normally you do not have to run the writer by hand.
`Nvdslogger unavailable` appears	The current system lacks the optional OpenTelemetry runtime library, so the course program skips the FPS logger; this does not affect the pose inference.
`Unable to acquire a user meta buffer` appears	It is a known Tracker warning of the current version and it does not block this case; do not blindly enlarge the metadata pool, as that would cause a performance drop.
The video `out.mp4` cannot be played	Check whether it was force-terminated before the EOS finalization; run it again and wait for `End of stream`.